

VIDEO NOTES · PART 9

Chains in LangChain

Sequential, parallel, router and custom workflows

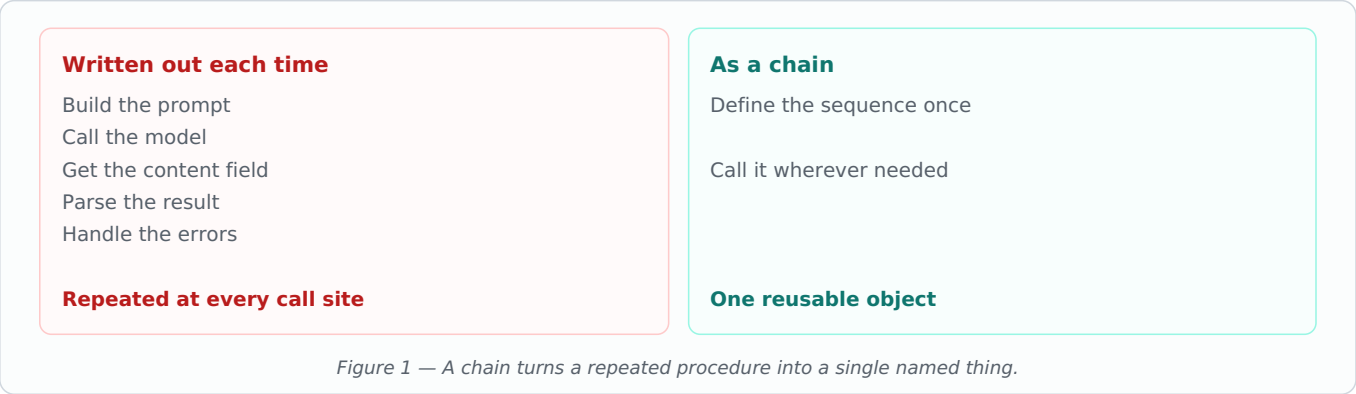
What this document is

A plain-English guide to the component LangChain is named after — how to link steps into one reusable workflow.

1 What a Chain Is

Almost nothing useful is a single model call. A realistic task involves building a prompt, calling a model, parsing the reply — and often feeding that result into another step entirely.

Written by hand, that sequence gets repeated everywhere it is needed. A **chain** packages the whole sequence into one object you can call like a single step.



This works because of **runnables** from earlier in the course. Prompts, models and parsers all share the same interface, so they connect directly — and the finished chain is itself a runnable, which is what allows chains to be nested inside other chains.

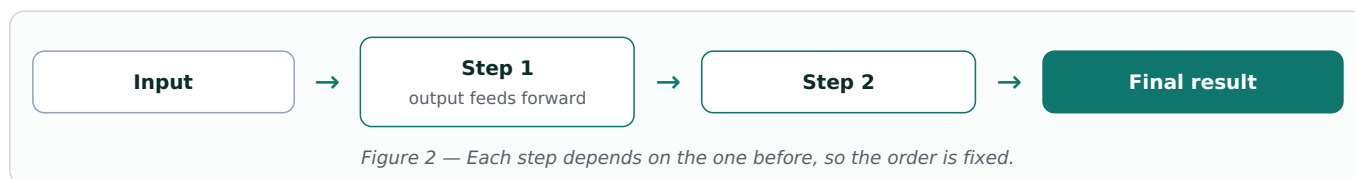
The property that makes chains scale: a chain can be a step in another chain. Any workflow you build stays available as a single building block for the next one.

The four shapes

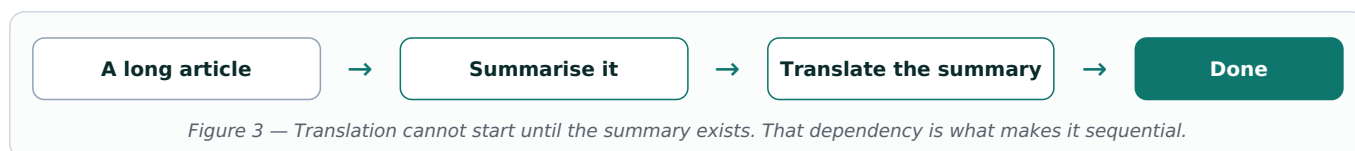
Shape	The question it answers
Sequential	What happens next?
Parallel	What can happen at the same time?
Router	Which path should this input take?
Custom	How do I combine the above for my actual problem?

2 Sequential Chains

The most common shape. Steps run in order, and each step's output becomes the next step's input.



A worked example



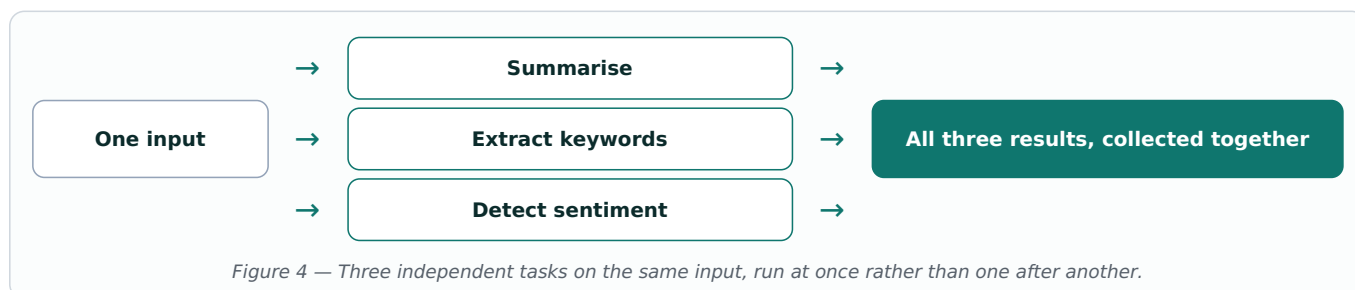
How to spot a sequential chain: ask whether step two needs step one's answer. If it does, the steps must run in order. If it does not, you probably want a parallel chain instead.

The cost of length: every step is a model call, so a five-step sequential chain is five times the latency and roughly five times the spend. Combine steps where one prompt could reasonably do the work of two.

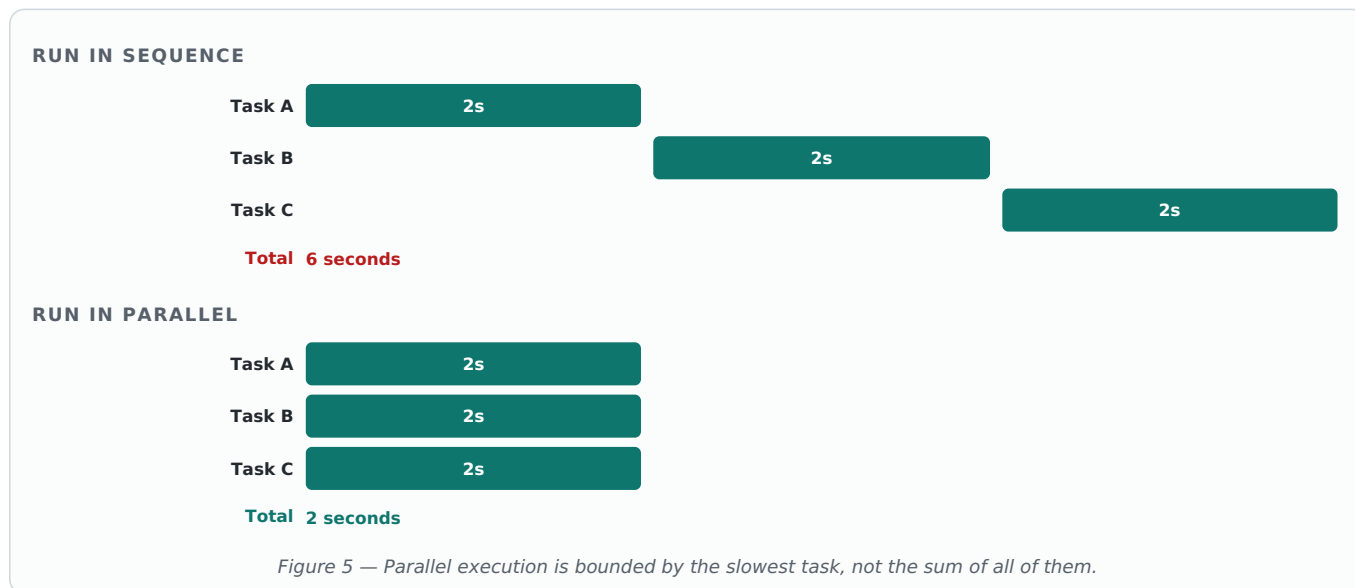
3 Parallel Chains

When several steps need the same input and none depends on another, running them in sequence wastes time.

RunnableParallel runs them simultaneously and collects the results together.



Where the time goes

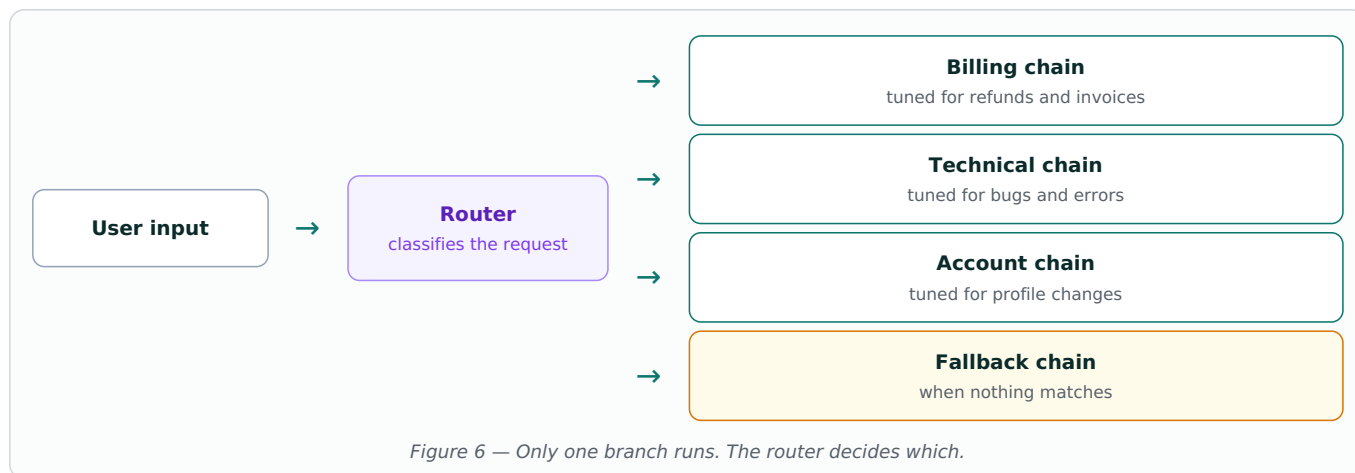


It saves time, not money. Three calls still cost three calls. Parallel chains reduce waiting, and they are also limited by provider rate limits — firing many requests at once can trip them.

4 Router Chains

Sequential and parallel chains both run a fixed set of steps. A **router chain** decides which path to take based on the input itself.

A first step classifies the request, then the router sends it to the specialised chain built for that kind of request.



Why route instead of using one big prompt

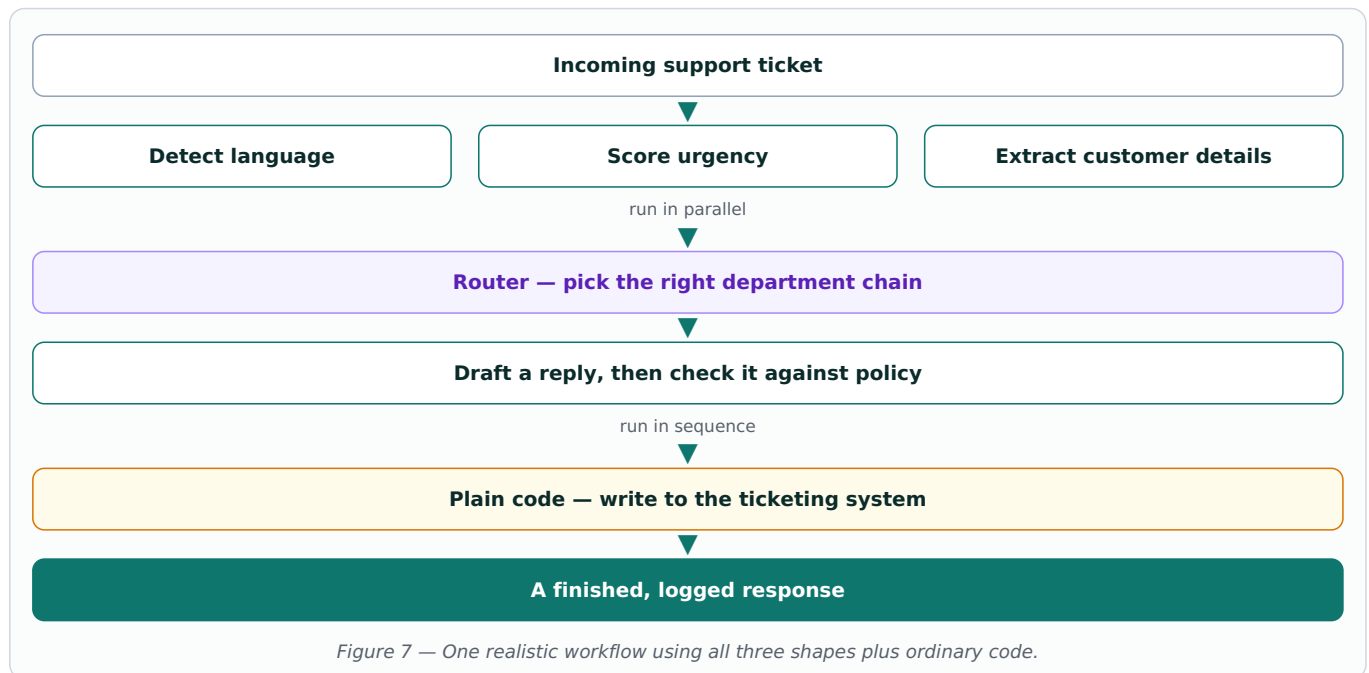
Benefit	What it means in practice
Focused prompts	Each branch handles one kind of request, so its prompt can be specific instead of covering every case at once.
Cheaper on average	A short specialised prompt costs less than one enormous prompt carrying instructions for every scenario.
Independent changes	Improving the billing branch cannot accidentally break technical support.
Mixed models	Simple branches can use a small fast model while complex ones use a stronger one.

Always include a fallback. Real users send things that fit no category. Without a default branch, an unmatched input has nowhere to go. Routing accuracy is also now part of your system's accuracy — a misrouted request gets a confident answer from entirely the wrong specialist.

The few-shot prompting from Part 4 is the usual tool for the classification step, since your categories are specific to your organisation.

5 Custom Chains

Real business workflows rarely match one shape exactly. A **custom chain** combines sequential, parallel and routing pieces — and can include ordinary code that is not a model call at all.



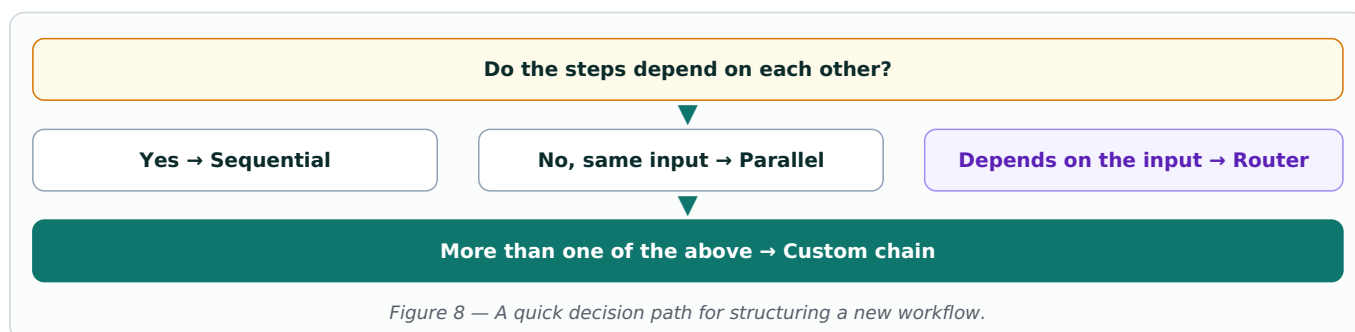
The point worth taking from this: the shapes are not competing options to choose between. They are pieces you assemble. Most production workflows are custom chains built from all of them.

Keeping custom chains manageable

- **Name each sub-chain** for what it does, not where it sits. "Urgency scorer" is useful; "step three" is not.
- **Build and test each piece alone** before wiring them together. Debugging a six-step chain as a single unit is painful.
- **Watch the total latency.** Sequential sections add up; move independent work into parallel sections wherever the dependencies allow.
- **Log at the joins.** When output looks wrong, you want to know which step produced it — this is what the logger from Part 3 was for.

6 Choosing a Shape

Shape	Use when	Watch out for
Sequential	Each step needs the previous step's output	Latency and cost multiply with every step added
Parallel	Several independent tasks share one input	Saves time but not money; watch rate limits
Router	Different inputs need genuinely different handling	Misclassification sends work to the wrong branch; always add a fallback
Custom	The real workflow needs more than one shape	Complexity grows quickly; test each piece on its own



How this connects to the rest of the course

Component	Its role inside a chain
Prompt templates	Build the input for each step
LLMs	The step that actually generates
Output parsers	Make each step's output safe to pass to the next
Memory	Supplies conversation history into the chain
Runnables	The shared interface that lets it all connect

Output parsers matter more inside chains. When a step's result feeds directly into the next step, loose free text propagates the problem forward. Parsing between steps is what keeps a long chain reliable.

Practice before the next lecture

- ☐ Build a two-step sequential chain and confirm the second step receives the first's output.
- ☐ Convert three independent calls into a parallel chain and time both versions.
- ☐ Write a router with three branches and a fallback, then send it an input that fits none.
- ☐ Nest a finished chain inside another chain as a single step.
- ☐ Add an output parser between two steps and observe what changes.
- ☐ Combine all three shapes into one custom chain for a workflow from your own work.

The short version: sequential for dependent steps, parallel for independent ones, routers for input-dependent paths,

and custom chains for real workflows — which are almost always a mixture of all three.